Name: Ryaan Devnani	Class: T1	Roll No: 2401023
Batch: T11	Subject: Agile Software Development & DevOps	Deptartment: AI & DS

Experiment no. 02

Aim: To containerise an application using Docker

LO 04: To automate provisioning, configuration, container orchestration, and cloud resource management using Ansible, Docker, Kubernetes, and Terraform.

Theory:

Docker
Docker is a platform/tool that lets you package an application along with everything it needs (code, libraries, dependencies, settings) into a single unit that can run the same way on any machine — your laptop, a server, the cloud, doesn't matter.

Containerization
This is the technique itself — packaging an application and its dependencies together into an isolated unit called a "container," so it runs consistently everywhere without "it works on my machine" problems. It's an alternative to running apps directly on a bare OS or inside a full virtual machine. Containers are lighter than VMs because they share the host machine's OS kernel instead of running a whole separate OS.

	Image	**Container**
What it is	A static, read-only template/blueprint	A running (or stopped) instance made from that image
State	Doesn't change, doesn't run by itself	Live, active, has a running process, can change while running
Analogy	A class in programming	An object/instance of that class
Example	ubuntu image sitting in docker images	The container 987804e55227 you created by running docker run ubuntu /bin/bash
Storage	Stored on disk as layers, reusable	Created from an image at runtime, has its own writable layer on top

PROCEDURE / COMMANDS PERFORMED

The following commands were executed in sequence in the command prompt (run as Administrator) to verify the Docker installation, search for a base image, pull it, and run a container from it.

#	Command	Purpose / What It Does
1	`docker --version`	Displays the installed Docker Engine version, confirming Docker is installed and accessible from the command line.
2	`docker images`	Lists all Docker images currently stored locally, showing repository name, tag, image ID, creation date, and size.
3	`docker search ubuntu`	Searches Docker Hub for images related to "ubuntu" and lists available repositories with their star ratings and official status.
4	`docker search ubuntu --filter stars=50`	Searches Docker Hub for ubuntu-related images, filtering results to show only those with 50 or more stars.
5	`docker pull ubuntu`	Downloads (pulls) the latest official Ubuntu image from Docker Hub to the local system.

#	Command	Purpose / What It Does
6	`docker images`	Re-run after the pull to verify that the ubuntu image now appears in the local image list.
7	`docker run ubuntu /bin/bash`	Creates and starts a new container from the ubuntu image and runs the /bin/bash shell inside it.
8	`docker ps`	Lists currently running containers, showing container ID, image used, command, status, ports, and name.

Explanation of the workflow

The Docker version was first verified to confirm a working installation (Docker version 25.0.3).

Existing local images were listed using docker images, showing previously pulled images such as redis, ubuntu, and alpine/git.

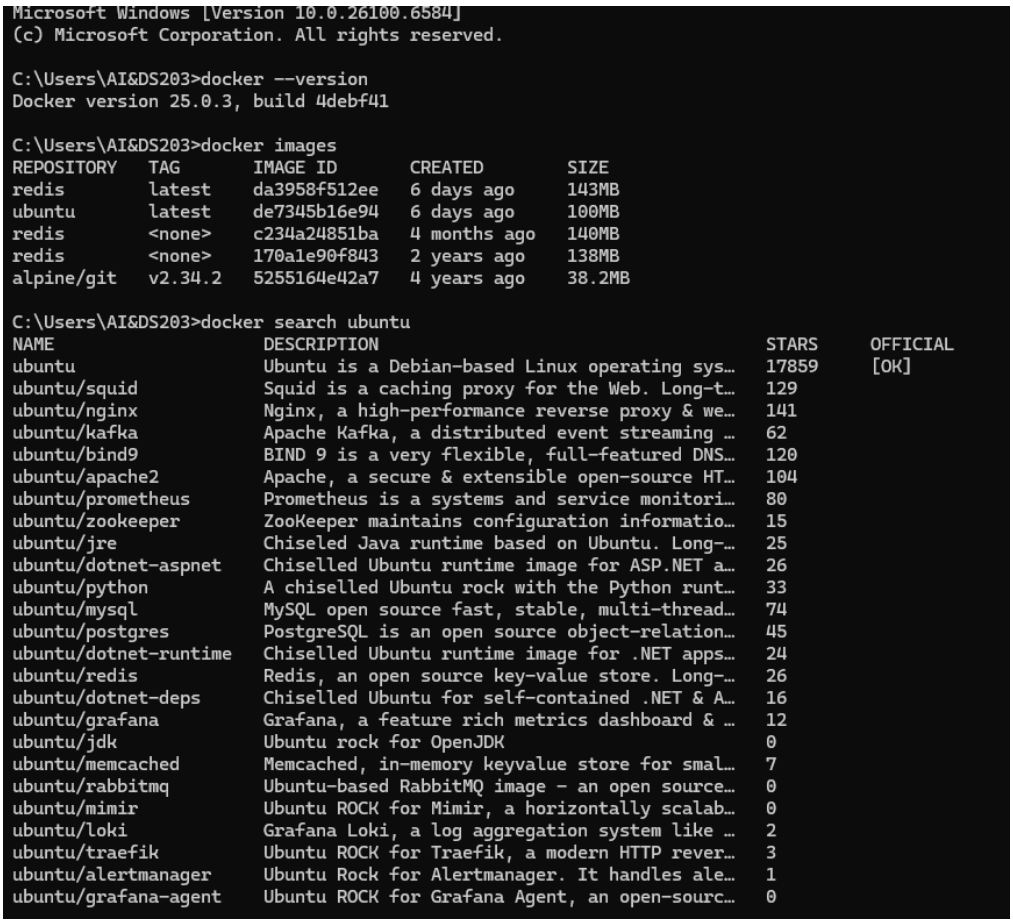
Docker Hub was searched for the "ubuntu" repository, returning a list of official and community-maintained ubuntu-based images along with their star counts.

The search was refined using the --filter stars=50 flag to narrow results to only the most popular/ trusted images.

The official ubuntu image was pulled from Docker Hub using docker pull ubuntu, and docker images was re-run to confirm the image (100MB) was now available locally.

A container was created and started from the ubuntu image using docker run ubuntu /bin/bash, and docker ps confirmed the container (named laughing_agnesi) was up and running.

Observations

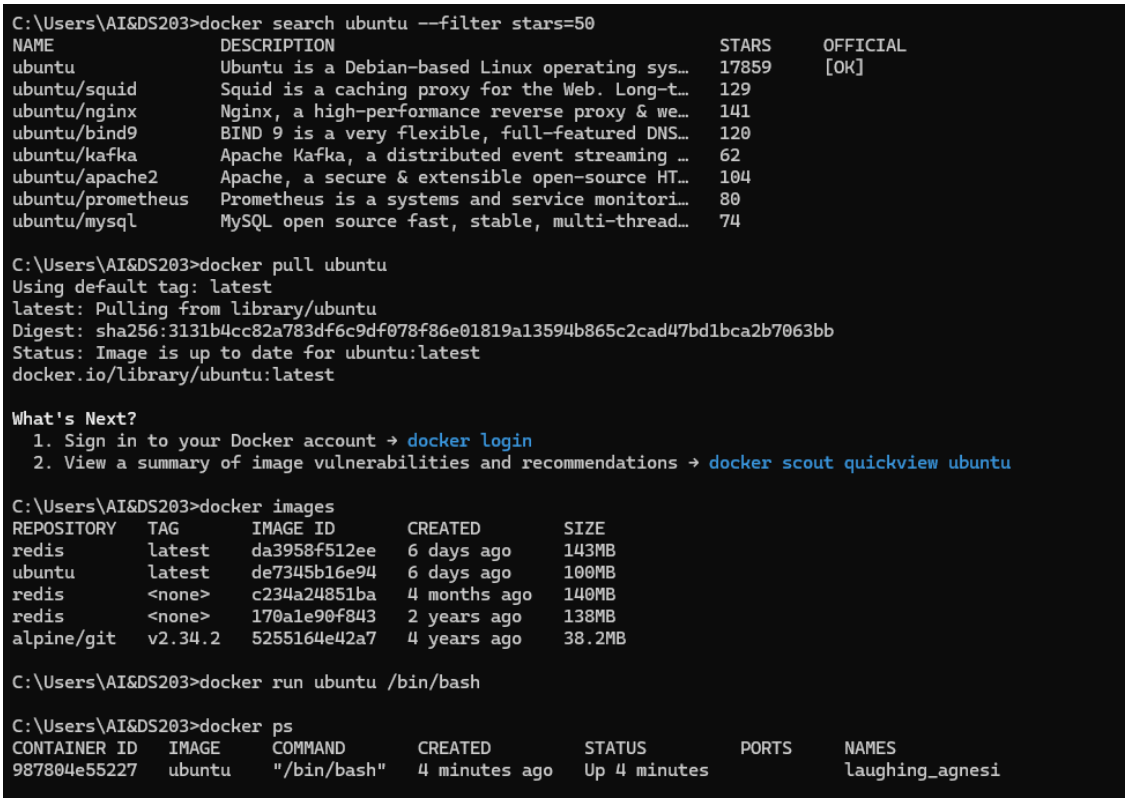


```
Microsoft Windows [Version 10.0.26100.6584]
(c) Microsoft Corporation. All rights reserved.

C:\Users\AI&DS203>docker --version
Docker version 25.0.3, build 4debf41

C:\Users\AI&DS203>docker images
REPOSITORY   TAG       IMAGE ID       CREATED        SIZE
redis        latest    da3958f512ee   6 days ago     143MB
ubuntu       latest    de7345b16e94   6 days ago     100MB
redis        <none>    c234a24851ba   4 months ago   140MB
redis        <none>    170a1e90f843   2 years ago    138MB
alpine/git   v2.34.2   5255164e42a7   4 years ago    38.2MB

C:\Users\AI&DS203>docker search ubuntu
NAME                      DESCRIPTION                                     STARS     OFFICIAL
ubuntu                    Ubuntu is a Debian-based Linux operating sys…   17859     [OK]
ubuntu/squid              Squid is a caching proxy for the Web. Long-t…   129
ubuntu/nginx              Nginx, a high-performance reverse proxy & we…   141
ubuntu/kafka              Apache Kafka, a distributed event streaming …   62
ubuntu/bind9              BIND 9 is a very flexible, full-featured DNS…   120
ubuntu/apache2            Apache, a secure & extensible open-source HT…   104
ubuntu/prometheus         Prometheus is a systems and service monitori…   80
ubuntu/zookeeper          ZooKeeper maintains configuration informatio…   15
ubuntu/jre                Chiseled Java runtime based on Ubuntu. Long-…   25
ubuntu/dotnet-aspnet      Chiselled Ubuntu runtime image for ASP.NET a…   26
ubuntu/python             A chiselled Ubuntu rock with the Python runt…   33
ubuntu/mysql              MySQL open source fast, stable, multi-thread…   74
ubuntu/postgres           PostgreSQL is an open source object-relation…   45
ubuntu/dotnet-runtime     Chiselled Ubuntu runtime image for .NET apps…   24
ubuntu/redis              Redis, an open source key-value store. Long-…   26
ubuntu/dotnet-deps        Chiselled Ubuntu for self-contained .NET & A…   16
ubuntu/grafana            Grafana, a feature rich metrics dashboard & …   12
ubuntu/jdk                Ubuntu rock for OpenJDK                         0
ubuntu/memcached          Memcached, in-memory keyvalue store for smal…   7
ubuntu/rabbitmq           Ubuntu-based RabbitMQ image - an open source…   0
ubuntu/mimir              Ubuntu ROCK for Mimir, a horizontally scalab…   0
ubuntu/loki               Grafana Loki, a log aggregation system like …   2
ubuntu/traefik            Ubuntu ROCK for Traefik, a modern HTTP rever…   3
ubuntu/alertmanager       Ubuntu Rock for Alertmanager. It handles ale…   1
ubuntu/grafana-agent      Ubuntu ROCK for Grafana Agent, an open-sourc…   0
```



ADDITIONAL COMMANDS TO EXPLORE

As instructed, the following additional Docker commands can be tried to further explore container lifecycle management, image handling, networking, and multi-container orchestration. Screenshots of these can be added to extend the observations section above.

#	Command	Purpose / What It Does
1	`docker exec -it 987804e55227 bash`	Opens an interactive terminal session inside the already-running ubuntu container.
2	`docker stats`	Displays a live stream of resource usage (CPU, memory, network) — run while the container is up, before stopping it.
3	`docker inspect 987804e55227`	Shows detailed low-level configuration and metadata about the running container in JSON format.
4	`docker logs 987804e55227`	Displays the console output/logs generated by the container.
5	`docker ps -a`	Lists all containers, including stopped ones, not just running containers.
6	`docker stop 987804e55227`	Gracefully stops the running container.
7	`docker start 987804e55227`	Restarts the previously stopped container.
8	`docker rm -f 987804e55227`	Force-stops (if needed) and removes the container in a single step.
9	`docker rmi <ubuntu_image_id>`	Removes the ubuntu image from local storage (image ID obtained from docker images).
10	`docker network ls`	Lists all Docker networks available on the system.
11	`docker volume ls`	Lists all Docker volumes used for persistent data storage.

#	Command	Purpose / What It Does
12	`docker system prune`	Removes unused containers, networks, and dangling images to free up disk space (confirm with 'y' when prompted).

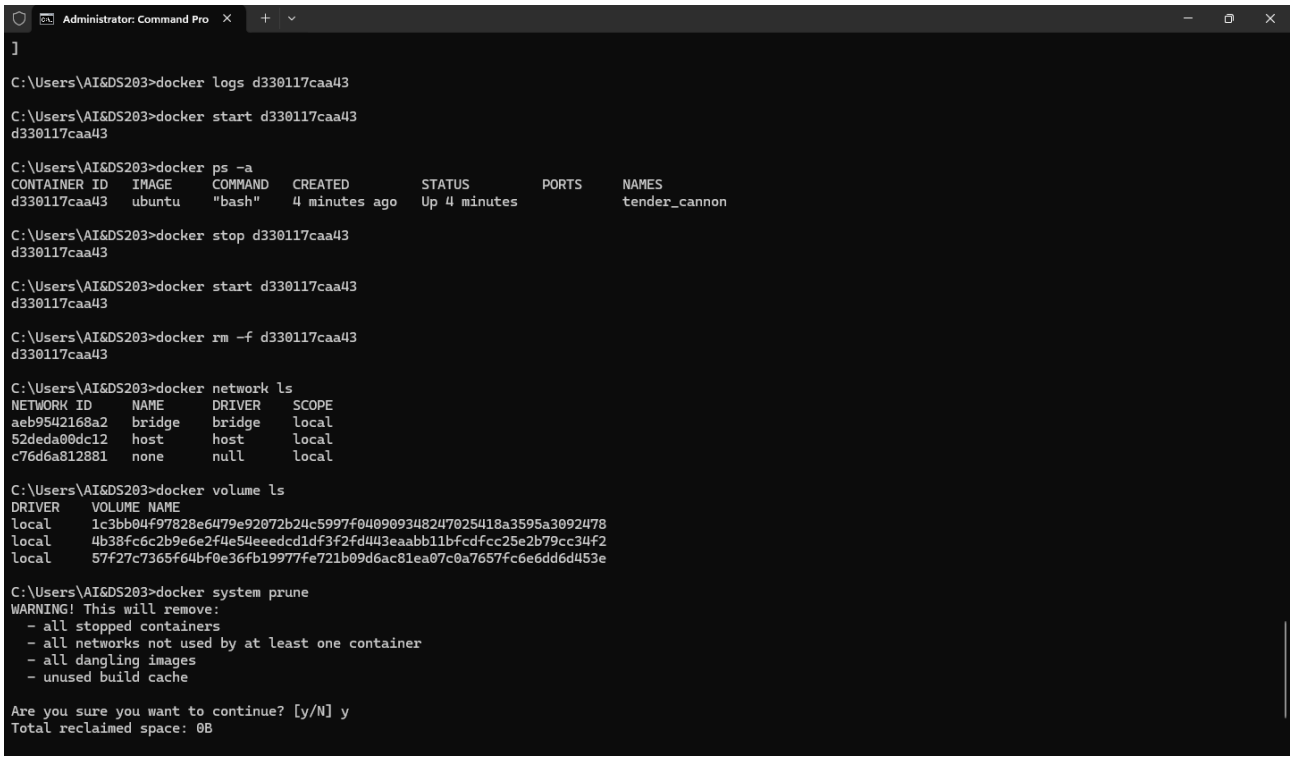
```
C:\Users\AI&DS203>docker exec -it d330117caa43 bash
root@d330117caa43:/# exit
exit

C:\Users\AI&DS203>docker stats --no-stream
CONTAINER ID   NAME           CPU %     MEM USAGE / LIMIT   MEM %     NET I/O       BLOCK I/O   PIDS
d330117caa43   tender_cannon   0.00%     904KiB / 7.618GiB   0.01%     1.09kB / 0B   0B / 0B     1

C:\Users\AI&DS203>docker inspect d330117caa43
[
    {
        "Id": "d330117caa43096d5b176d5111b08109554ffd942d792c6a37e188ff82b71e3b",
        "Created": "2026-07-20T11:05:14.119183126Z",
        "Path": "bash",
        "Args": [],
        "State": {
            "Status": "running",
            "Running": true,
            "Paused": false,
            "Restarting": false,
            "OOMKilled": false,
            "Dead": false,
            "Pid": 1585,
            "ExitCode": 0,
            "Error": "",
            "StartedAt": "2026-07-20T11:05:14.513382404Z",
            "FinishedAt": "0001-01-01T00:00:00Z"
        },
        "Image": "sha256:de7345b16e942e22044c6ba053020ec85ae879984860a9918517d54eb6cef851",
        "ResolvConfPath": "/var/lib/docker/containers/d330117caa43096d5b176d5111b08109554ffd942d792c6a37e188ff82b71e3b/resolv.conf",
        "HostnamePath": "/var/lib/docker/containers/d330117caa43096d5b176d5111b08109554ffd942d792c6a37e188ff82b71e3b/hostname",
        "HostsPath": "/var/lib/docker/containers/d330117caa43096d5b176d5111b08109554ffd942d792c6a37e188ff82b71e3b/hosts",
        "LogPath": "/var/lib/docker/containers/d330117caa43096d5b176d5111b08109554ffd942d792c6a37e188ff82b71e3b/d330117caa43096d5b176d5111b08109554ffd942d79
2c6a37e188ff82b71e3b-json.log",
        "Name": "/tender_cannon",
        "RestartCount": 0,
        "Driver": "overlay2",
        "Platform": "linux",
```

```
Administrator: Command Pro
        },
        "NetworkSettings": {
            "Bridge": "",
            "SandboxID": "1f659eeadfb4776fc020541146953325ca2825819f47ed367ea495b35edbd53e",
            "SandboxKey": "/var/run/docker/netns/1f659eeadfb4",
            "Ports": {},
            "HairpinMode": false,
            "LinkLocalIPv6Address": "",
            "LinkLocalIPv6PrefixLen": 0,
            "SecondaryIPAddresses": null,
            "SecondaryIPv6Addresses": null,
            "EndpointID": "bd200d5181a2f78e3d5cf01a0eba7deecc2ee2876f9587a4070e88badedb61f7",
            "Gateway": "172.17.0.1",
            "GlobalIPv6Address": "",
            "GlobalIPv6PrefixLen": 0,
            "IPAddress": "172.17.0.2",
            "IPPrefixLen": 16,
            "IPv6Gateway": "",
            "MacAddress": "02:42:ac:11:00:02",
            "Networks": {
                "bridge": {
                    "IPAMConfig": null,
                    "Links": null,
                    "Aliases": null,
                    "MacAddress": "02:42:ac:11:00:02",
                    "NetworkID": "aeb9542168a2864e7720585e6f8e5e49437b47dc69e9e1b3ff5ee34271bc5808",
                    "EndpointID": "bd200d5181a2f78e3d5cf01a0eba7deecc2ee2876f9587a4070e88badedb61f7",
                    "Gateway": "172.17.0.1",
                    "IPAddress": "172.17.0.2",
                    "IPPrefixLen": 16,
                    "IPv6Gateway": "",
                    "GlobalIPv6Address": "",
                    "GlobalIPv6PrefixLen": 0,
                    "DriverOpts": null,
                    "DNSNames": null
                }
            }
        }
    }
]

C:\Users\AI&DS203>docker logs d330117caa43
```



CONCLUSION

Through this experiment, Docker was successfully installed and verified, and the concept of containerisation was studied and practically implemented. An official Ubuntu base image was searched for, filtered, and pulled from Docker Hub, and a container was successfully created and run from this image. The docker ps command confirmed that the container was active and running. This experiment demonstrates how Docker simplifies application deployment by packaging an OS environment and its dependencies into a lightweight, portable, and reproducible container, thereby overcoming environment-inconsistency issues faced in traditional deployment methods. Further exploration with commands for interactive container access (exec), resource monitoring (stats, inspect, logs), full lifecycle management (stop, start, remove), and cleanup (rmi, network ls, volume ls, system prune) will provide a deeper understanding of the complete Docker ecosystem.